

Project Algorithms & Machine Learning — Deep Dive

CASSA RA 2026-27 · every algorithm in your CV, explained with theory + math + example

Written from scratch: even if you have never studied an algorithm formally, this explains what it does, the one key equation, and a tiny example. Covers your samplers (emcee, MultiNest, dynesty, Nautilus), gradient boosting (PIE-ADA), CNNs (RGC), and VAE + Transformer (LunaXNet), plus the ML ideas you must be able to defend.

PART 1 — The forward model you are fitting (FSPS)

Before any sampler, you need a **model that predicts data from parameters**. - **Stellar Population Synthesis (FSPS)**: given a **star-formation history (SFH)**, **age**, **metallicity**, **dust**, and an **IMF**, it adds up the light of all the stars → a predicted galaxy **SED** (flux in each band). - Schematically: $\text{model_flux}(\text{band}) = \int \text{SFH}(t) \times \text{SSP}(t, Z) dt$, then redden by dust and redshift. - Your job: find the parameters whose predicted SED best matches the **observed** `ugriz` (SDSS) or `ugriYJH` (CSP) photometry.

PART 2 — Bayesian inference (the framework)

Posterior $\propto$ Likelihood $\times$ Prior, i.e. $P(\theta|D) \propto P(D|\theta) \cdot P(\theta)$. - **Likelihood** (Gaussian errors): $\ln L = -\frac{1}{2} \sum_{\text{bands}} [(f_{\text{obs}} - f_{\text{model}})^2 / \sigma^2]$. - **Prior** $P(\theta)$: what you allow before seeing data (age 0 to age of Universe, dust ≥ 0 , etc.). - **Posterior**: the answer, a probability distribution over parameters. You report the **median + credible interval**, not one best-fit. - **Evidence** $Z = \int L(\theta) P(\theta) d\theta$: the total probability of the data; used to compare whole models.

Two job families: **MCMC** samples the posterior shape; **nested sampling** also computes the evidence Z .

PART 3 — MCMC samplers

3.1 Metropolis-Hastings (the core idea)

A random walk that visits parameter values **in proportion to their probability**. - Propose a step θ' . Accept it with probability $\alpha = \min(1, P(\theta'|D) / P(\theta|D))$. - If the new point is more probable, always go; if less probable, sometimes go. Over time the chain's histogram = the posterior. - *Tiny example*: if θ' is half as probable as θ , $\alpha = 0.5$, so you accept it 50% of the time. This lets the walker explore, not just climb.

3.2 emcee (affine-invariant ensemble)

- Runs many **walkers** at once. Each walker moves by being "stretched" toward another walker (the **stretch move**), so it adapts to the posterior shape automatically, no hand-tuned step size.
- **Strength**: robust, simple. **Weakness**: slow to converge for expensive likelihoods; gives no evidence Z .
- **Health checks**: **autocorrelation time** (steps between independent samples), burn-in, chain mixing.

PART 4 — Nested sampling (why your speed result exists)

4.1 The idea

Instead of walking, **peel the prior like an onion** from low to high likelihood. - Keep N **live points** drawn from the prior. Repeatedly remove the **lowest-likelihood** one, record it, and replace it with a new point of **higher** likelihood. - Each step shrinks the remaining prior volume by roughly $X \rightarrow X \cdot N / (N+1)$. - The evidence is the sum $Z \approx \sum L_i \cdot \Delta X_i$ (likelihood $\times$ the shell of volume it covers). The posterior comes out as a by-product.

4.2 The three you benchmarked

- **MultiNest:** bounds the live points with **overlapping ellipsoids**; handles **multimodal / bimodal** posteriors well (your CSP fallback).
- **dynesty:** pure-Python **dynamic** nested sampling; adds live points where the posterior needs resolution.
- **Nautilus: importance nested sampling boosted by a neural network.** The NN learns the shape of the high-likelihood region, so proposals land in good places $\rightarrow$ **far fewer likelihood evaluations**.

4.3 Why Nautilus is $\sim 60\times$ faster (say this in the viva)

The cost is dominated by likelihood calls (each is a full FSPS SED evaluation). emcee must random-walk to convergence, wasting many calls. Nautilus learns where the posterior mass is and samples there, so it needs roughly $60\times$ fewer calls for the same answer. In your benchmark the recovered ages differ by only $+0.64 \pm 0.29$ Gyr (2.21σ), so the speed-up costs almost no accuracy.

PART 5 — Gradient boosting / LightGBM (PIE-ADA)

5.1 What boosting is

Build an **ensemble of small decision trees, added one at a time**, where each new tree fixes the errors of the ones so far. - Model after m trees: $F_m(x) = F_{\{m-1\}}(x) + \gamma \cdot h_m(x)$. - Each new tree h_m is fit to the **negative gradient of the loss** (the "pseudo-residuals"), i.e. it learns the direction that most reduces the error. That is why it is called *gradient* boosting. - *Intuition:* tree 1 makes a rough guess; tree 2 predicts tree 1's mistakes; tree 3 predicts what is still wrong; and so on.

5.2 What makes LightGBM fast (good detail to drop)

- **Histogram binning:** groups continuous features into bins $\rightarrow$ far fewer split points to test.
- **Leaf-wise (best-first) growth:** splits the leaf that reduces loss most, not level-by-level $\rightarrow$ deeper, more accurate trees for the same budget.
- Handles large tabular data (your 271 features, 1.4M observations) efficiently.

5.3 The loss it minimised (your metric)

Multiclass **weighted log-loss**: $\text{LogLoss} = - (1/N) \sum_i \sum_c w_c \cdot y_{\{ic\}} \cdot \ln(p_{\{ic\}}) - p_{\{ic\}}$ = predicted probability that object i is class c ; $y_{\{ic\}} = 1$ if true. - It **punishes confident wrong answers** heavily and the weights w_c stop common classes from dominating rare ones. Your result: **0.5763 ± 0.0083** , 80.33% accuracy.

PART 6 — Convolutional Neural Networks (RGC)

6.1 How a CNN sees an image

- A **convolution** slides a small **kernel** (e.g. 3×3 weights) across the image, computing dot products $\rightarrow$ a **feature map** that lights up where a pattern (edge, blob, jet) appears.

- **ReLU** $f(x) = \max(0, x)$ adds non-linearity; **pooling** shrinks the map (keeps the strongest response) → the network builds from edges → shapes → whole morphologies (FR I vs FR II).
- **Training**: compare output to the label, compute loss, and **backpropagation** adjusts every weight by gradient descent.

6.2 Two terms on your CV

- **Semi-supervised learning**: RGC uses a **small labelled set + a large unlabelled set** (radio surveys have far more images than labels). The model uses its own confident predictions (pseudo-labels) / consistency to learn from unlabelled data.
- **Grad-CAM**: produces a **heatmap of which pixels drove the classification** → shows the network looked at the real radio lobes, not artefacts. This is your interpretability tool.

PART 7 — VAE + Transformer (LunaXNet)

7.1 Autoencoder → Variational Autoencoder (VAE)

- An **autoencoder** squeezes input through a bottleneck: **encoder** → small **latent code** z → **decoder** rebuilds the input. The code captures the essence.
- A **VAE** makes the latent space a **probability distribution** (mean + variance), so it is smooth and generative. It is trained on the **ELBO**: $ELBO = E[\ln p(x|z)] - KL(q(z|x) \parallel p(z)) =$ **reconstruction quality** minus a **KL penalty** that keeps the latent code close to a standard Gaussian.
- **Reparameterization trick**: $z = \mu + \sigma \cdot \epsilon$ ($\epsilon \sim \text{Normal}$) lets gradients flow through the random sampling.
- **CVAE (conditional)**: condition on extra input (e.g. spatial context) so generation/segmentation is guided.

7.2 Transformer & self-attention (why for 85 hyperspectral bands)

- **Self-attention** lets every element look at every other and weight what matters:
 $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$.
- Q, K, V are learned projections; the softmax gives **weights** for how much each band attends to each other band; $\sqrt{d}$ keeps the scores stable.
- **Why here**: the 85 M3 spectral bands are a **sequence**; mineral absorption features are relationships *between distant bands*. Attention captures those long-range links that a local CNN kernel cannot. Hence the **hybrid CNN + Transformer** (CNN for spatial texture, Transformer for spectral relationships).

PART 8 — General ML understanding (defend these)

- **Overfitting vs underfitting / bias-variance tradeoff**: a model too simple misses the pattern (high bias); too complex memorises noise (high variance). You balance with regularisation, more data, and validation.
- **Train / validation / test split & cross-validation**: never judge on training data; k-fold CV gives a stable estimate (your log-loss ± 0.0083 is a CV spread).
- **Class imbalance**: rare classes need reweighting / careful metrics (why weighted log-loss).
- **Metrics**: accuracy (can mislead when imbalanced), **precision / recall / F1, ROC-AUC, confusion matrix**, and **log-loss** (probability quality).
- **Regularisation**: L1/L2 penalties, dropout, early stopping, tree depth limits.
- **Supervised vs unsupervised vs semi-supervised**: labels for all / none / some (RGC = some).

Quick-recall (flash-card)

- **Bayes**: $\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$; $\ln L = -\frac{1}{2} \sum (\Delta f / \sigma)^2$; evidence $Z = \int L \pi d\theta$.
- **Metropolis**: accept $\alpha = \min(1, P'/P)$. **emcee**: affine-invariant walkers, no Z, check autocorr time.

- **Nested sampling:** peel prior by live points, $Z \approx \sum L_i \Delta X_i$. MultiNest (ellipsoids, multimodal), dynesty (dynamic), **Nautilus (NN importance NS $\rightarrow \sim 60\times$ fewer likelihood calls)**.
- **Gradient boosting:** $F_m = F_{m-1} + \gamma h_m$, trees fit negative gradient. LightGBM: histogram bins + leaf-wise growth. **Weighted log-loss** $-(1/N) \sum w_c y \ln p = 0.5763$.
- **CNN:** kernels $\rightarrow$ feature maps $\rightarrow$ ReLU $\rightarrow$ pooling $\rightarrow$ backprop. Semi-supervised (few labels), Grad-CAM (pixel heatmap).
- **VAE:** encoder $\rightarrow$ latent $z \rightarrow$ decoder; ELBO = reconstruction - KL; reparam $z = \mu + \sigma \epsilon$. **Attention:** $\text{softmax}(QK^T/\sqrt{d})V$. Hybrid CNN+Transformer for 85 spectral bands.
- **ML:** bias-variance, overfitting, CV, class imbalance, precision/recall/F1/AUC, regularisation.

Likely viva Qs

1. In plain terms, why is Nautilus $\sim 60\times$ faster than emcee?
2. What is nested sampling and what does it give that MCMC does not?
3. What is gradient boosting, and why LightGBM for PIE-ADA?
4. What does weighted log-loss measure and why use it?
5. How does a CNN classify a radio galaxy? What is Grad-CAM?
6. Why a Transformer for hyperspectral bands? What is self-attention?
7. How do you avoid overfitting and judge a model honestly?